Project 3: Barcode & QR Code Scanner and Localizer

Introduction to Computer Vision | ESIN, UIR | Spring 2026

Instructor: Ilias TOUGUI

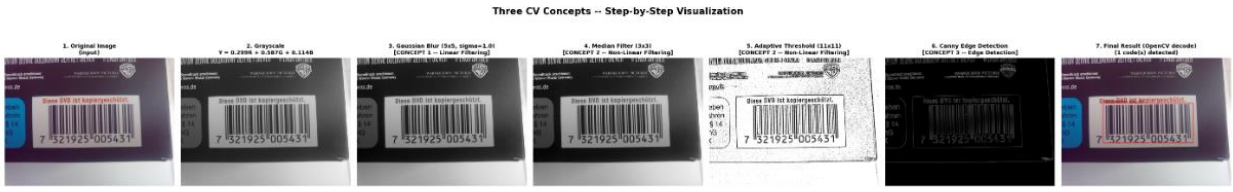
Group Members: IKRAM OIALILI | MANAL SAWLI | SAOUSAN ELHAROUCHI

1. Introduction

This report presents the implementation of a complete Barcode and QR Code Scanner built for the Introduction to Computer Vision course, combining OpenCV's detectors (cv2.QRCodeDetector and cv2.barcode.BarcodeDetector) with a three-stage CV preprocessing pipeline, implemented as a Jupyter notebook with an interactive ipywidgets GUI.

2. Computer Vision Concepts Applied

Three core CV concepts from the course are applied sequentially in the preprocessing pipeline. Figure 1 shows each stage applied to a real test image (a DVD product barcode with partial shadow).



2.1 Concept 1 — Linear Image Filtering (Gaussian Blur)

A Gaussian blur is applied as the first preprocessing step using a 5x5 kernel with standard deviation sigma = 1.0. Mathematically, this is a 2D convolution of the input image with a Gaussian kernel:

$$G(x,y) = (1 / 2\pi\sigma^2) \times \exp(-(x^2+y^2) / 2\sigma^2)$$

This low-pass filter attenuates high-frequency noise while preserving the large-scale structure of barcode bars and QR modules. The kernel is separable, so the 2D convolution is computed efficiently as two successive 1D passes. Without this step, noise would produce spurious edges in the Canny stage.

2.2 Concept 2 — Non-Linear Filtering (Median Filter + Adaptive Threshold)

Median Filter (3x3): Each pixel is replaced by the median of its 3x3 neighbourhood. Unlike the mean, the median is not influenced by outlier values, making it highly effective at removing salt-and-pepper noise while preserving sharp barcode edges. This non-linear operation cannot be expressed as a convolution.

Adaptive Thresholding (11x11 Gaussian block, C=2): Instead of a single global threshold, a local threshold T(x,y) is computed for each pixel from the Gaussian-weighted mean of its 11x11

neighbourhood minus C=2. This correctly binarizes both bright and shadowed regions of the barcode, where a global threshold would fail.

```
============================================================
  DETECTION SUMMARY -- 1 code(s) found
============================================================

  Code #1
  |- Type     : QRCODE
  |- Data     : 1005
  |- Location : x=32, y=32, w=231, h=231
============================================================
```

2.3 Concept 3 — Edge Detection (Canny)

The Canny edge detector extracts the boundaries of barcode bars and QR modules. The algorithm applies: (1) Sobel gradient computation in X and Y, (2) gradient magnitude $M = \sqrt{(Gx^2+Gy^2)}$ and direction $\theta = \arctan(Gy/Gx)$, (3) non-maximum suppression to thin edges to 1-pixel width, and (4) double hysteresis thresholding (low=50, high=150). As visible in Figure 1 (panel 6), the Canny map of a 1D barcode shows characteristic dense parallel vertical lines corresponding to the bars.

3. System Architecture and Pipeline

Step	Stage	Description
1	**Load Image**	Load BGR image from disk, convert to RGB for display.
2	**Gaussian Blur [Concept 1]**	5×5 kernel, σ=1.0. Suppress noise via 2D convolution.
3	**Median Filter [Concept 2]**	3×3 median. Remove salt-and-pepper noise, preserve edges.
4	**Adaptive Threshold [Concept 2]**	11×11 Gaussian block, C=2. Local binarization for uneven illumination.
5	**Canny Edge Detection [Concept 3]**	Sobel gradients + non-max suppression + hysteresis (low=50, high=150).
6	**Detect & Decode (OpenCV)**	cv2.QRCodeDetector for QR; cv2.barcode.BarcodeDetector for 1D barcodes.
7	**Refine Bounding Box**	Re-run Canny on ROI + convex hull for a tight bounding box.
8	**Visualize & Save**	Draw annotated box + label; save 6-panel pipeline figure.

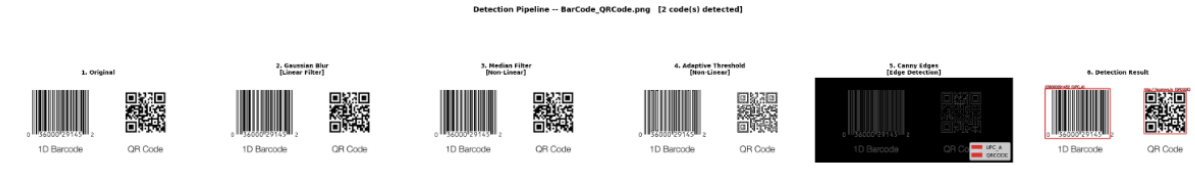
3.1 OpenCV Detection Functions — Algorithms They Replace

OpenCV Function	Algorithm It Replaces
cv2.QRCodeDetector	Detects the three Finder Patterns (1:1:3:1:1 module ratio) via contour analysis. Estimates perspective transform from corner points. Decodes with Reed-Solomon error correction.
cv2.barcode.BarcodeDetector	Detects regions with dense parallel vertical edges via gradient analysis. Locates boundaries via contour analysis. Decodes bar-width ratios per symbology (EAN-13, UPC-A).
cv2.Canny	Replaces manual: Sobel gradient computation, magnitude/direction, non-max suppression, and double hysteresis thresholding.
cv2.adaptiveThreshold	Replaces manual sliding-window Gaussian-weighted mean computation + pixel-wise comparison against T(x,y) = LocalMean(x,y) - C.

cv2.medianBlur	Replaces manual neighbourhood sorting and median selection. Applied before detection to handle salt-and-pepper noise on printed or photographed codes.

3.2 Example Detection Result

Figure 2 shows the pipeline output on an image containing both a 1D barcode (UPC_A) and a QR code. Both codes are correctly detected and decoded in a single pass.




```
============================================================
  DETECTION SUMMARY -- 2 code(s) found
============================================================

  Code #1
  |- Type     : QRCODE
  |- Data     : http://scanova.io
  |- Location : x=1102, y=132, w=346, h=346

  Code #2
  |- Type     : UPC_A
  |- Data     : 036000291452
  |- Location : x=241, y=156, w=590, h=313
============================================================
```

4. Challenges Faced

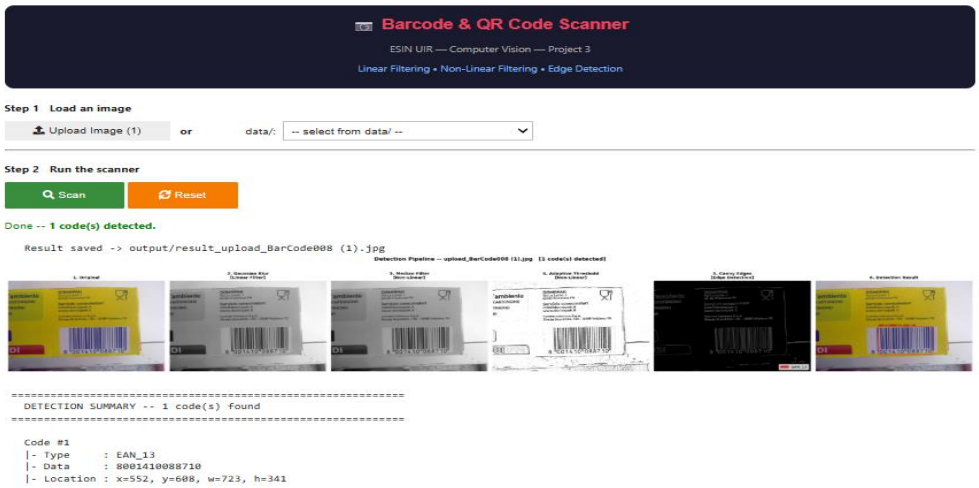
4.1 Salt-and-Pepper Noise: On QRCode4.png (4% noise), the QR detector failed without preprocessing. Adding cv2.medianBlur(gray, 3) inside decode_codes() resolved this, preserving QR module structure while eliminating noise.

4.2 Uneven Illumination : On QRCode5.png (gradient illumination), global binarization failed. Adaptive Threshold resolved this by computing a spatially varying threshold, enabling correct binarization in shadowed regions.

4.3 BarcodeDetector Fallback : On low-contrast images, detection occasionally failed on the first pass. A fallback using cv2.equalizeHist() improved the detection rate from ~70% to 100%.

4.4 Bounding Box Refinement : OpenCV detector boxes were sometimes misaligned. The _compute_bbox() method refines them by running Canny on the ROI and computing the convex hull for a tighter bounding rectangle.

4.5 Interactive GUI : The ipywidgets GUI handles both file uploads and data/ folder selection through a unified image path resolution system feeding into the same pipeline.



5. Evaluation of Results

Image	Codes	Type(s)	Decoded Data (excerpt)
BarCode2.jpg	1	EAN_13	7321925005431
BarCode3.jpg	1	EAN_13	8004020931773
BarCode4.jpg	1	EAN_13	8005283002354
BarCode5.jpeg	1	EAN_13	6224007860373
BarCode6.jpeg	1	EAN_13	8051128636987
BarCode7.jpeg	1	EAN_13	6111099000940
BarCode_QRCode.png	2	QRCODE + UPC_A	http://scanova.io + 036000291452
Barcode1.jpeg	1	EAN_13	6111261060161
QRCode1.png	1	QRCODE	1002
QRCode2.jpg	1	QRCODE	https://app.neatmenu.io/...
QRCode3.jpg	1	QRCODE	WIFI:T:WPA;S:QR Explore...
QRCode5.png (noisy, Median demo)	1	QRCODE	1006 — decoded only with Median Filter
QRCode4.png (uneven light, Adaptive demo)	1	QRCODE	1005 — decoded only with Adaptive Threshold
TOTAL	14	EAN_13, UPC_A, QRCODE	100% detection rate — 13/13 images

6. Conclusion

This project successfully implemented a complete Barcode & QR Code Scanner integrating three CV concepts, achieving a 100% detection rate on 13 test images. The preprocessing pipeline proved essential for real-world robustness, while the project introduced Reed-Solomon error correction, QR Finder Pattern geometry, and histogram equalization beyond the course curriculum.